

Comprehensive step-by-step guide for performing an **Azure SQL Managed Instance (MI) Health Check** — focused on **optimal performance, reliability, and cost efficiency**:

Azure SQL Managed Instance Health Check — Key Steps for Optimal Performance

1 Environment Overview

Before diving into performance metrics, understand the environment:

- **Instance Type & Tier:** General Purpose / Business Critical
- **vCore Configuration:** CPU, Memory ratio per tier
- **Storage Type & Size:** Premium SSD vs Azure Premium Fileshares
- **Azure Region:** Latency and availability considerations
- **Backup & DR Setup:** Auto backups, geo-redundant storage, failover groups

 **Tools:** Azure Portal → Overview + Metrics + SQL Assessment API

2 Connectivity & Network Configuration

- Verify **VNet integration, DNS resolution, and NSG rules**
- Check for **latency between app and MI** using ping or az network watcher test-connectivity
- Confirm **Private Endpoint** usage to avoid public exposure
- Review **firewall and subnet isolation** for performance consistency

 **Check in Portal:**

Resource → Networking → Connection type + Private endpoint + Subnets

3 Instance Resource Utilization

Monitor:

- **CPU usage trends**
- **Memory consumption (Buffer pool, plan cache)**
- **IOPS (reads/writes per second)**
- **TempDB utilization and contention**

 **Query Example:**

```
SELECT
    sqlserver_start_time,
    cpu_count,
    total_physical_memory_kb/1024 AS TotalMemoryMB,
    committed_kb/1024 AS UsedMemoryMB
FROM sys.dm_os_sys_info;
```

 **Best Practice:** Keep average CPU < 70% and plan cache hit ratio > 90%.

4 Database Performance Metrics

Use **Dynamic Management Views (DMVs)**:

- Identify top resource-consuming queries:

```
SELECT TOP 10
    qs.total_worker_time/1000 AS CPU_ms,
    qs.execution_count,
    qs.total_logical_reads,
    qs.total_logical_writes,
```

```
SUBSTRING(qt.text,1,100) AS QueryText
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
ORDER BY CPU_ms DESC;
```

- Review **Wait Stats** to identify bottlenecks:

```
SELECT TOP 10 wait_type, wait_time_ms/1000 AS WaitTimeSec, signal_wait_time_ms/1000 AS SignalWaitSec
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN ('SLEEP_TASK','SLEEP_SYSTEMTASK','SQLTRACE_BUFFER_FLUSH')
ORDER BY WaitTimeSec DESC;
```

Common Waits:

- PAGEIOLATCH_* → Slow storage I/O
- CXPACKET → Parallelism tuning
- RESOURCE_SEMAPHORE → Memory grants

5 Index & Statistics Health

- Identify **missing or unused indexes**
- Rebuild or reorganize **fragmented indexes** (>30%)
- Ensure **auto-update statistics** is ON

Sample Script:

```
SELECT
    dbschemas.[name] AS SchemaName,
    dbtables.[name] AS TableName,
    dbindexes.[name] AS IndexName,
    indexstats.avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats (DB_ID(), NULL, NULL, NULL, NULL) indexstats
INNER JOIN sys.tables dbtables ON dbtables.[object_id] = indexstats.[object_id]
INNER JOIN sys.schemas dbschemas ON dbtables.[schema_id] = dbschemas.[schema_id]
INNER JOIN sys.indexes dbindexes ON dbindexes.[object_id] = indexstats.[object_id]
AND indexstats.index_id = dbindexes.index_id
WHERE indexstats.avg_fragmentation_in_percent > 30
ORDER BY indexstats.avg_fragmentation_in_percent DESC;
```

6 Query Store & Workload Insights

- Enable **Query Store** if not active.
- Review **regressed queries** (those whose performance worsened after plan changes).
- Identify **parameter sniffing** issues.

In Azure Portal:

SQL Managed Instance → Databases → Query Performance Insight

 **Tip:** Use FORCE LAST GOOD PLAN for stabilized workloads.

7 TempDB Configuration

- Verify **number of TempDB files** = logical cores (up to 8 recommended)
- Monitor for **TempDB contention**
- Ensure **TempDB size auto-growth** is sufficient and not frequent



Script:

USE tempdb;

EXEC sp_helpfile;

8 Maintenance & Jobs

- Validate **Automated Backups & Retention policies**
- Check **index and statistics maintenance schedules**
- Ensure **DBCC CHECKDB** is regularly executed



Check via: Azure Automation or Managed Instance Agent Jobs

9 Alerts & Monitoring

- Use **Azure Monitor** or **Log Analytics** to track:
 - DTU/vCore utilization
 - Query duration spikes
 - Storage thresholds
- Set **Action Groups** for alerts on:
 - CPU > 85%
 - IO latency > 20 ms
 - Database size > 90%



Recommended Tools:

- Azure Monitor
- Log Analytics
- Azure SQL Analytics (Power BI integration)

10 Cost & Scalability Optimization

- Right-size compute (vCores) and storage based on usage patterns.
- Enable **auto-pause/resume** for Dev/Test MIs.
- Consider **Azure Hybrid Benefit** and **Reserved Instances** for cost reduction.
- Review **TempDB and backup storage costs**.



Deliverable Summary — Health Check Report Sections

1. Environment Overview
2. Connectivity & Network Assessment
3. Resource Utilization Summary
4. Query & Wait Stats Analysis
5. Index & Statistics Review
6. TempDB & Maintenance Health
7. Backup, DR & Security Review
8. Monitoring & Alerting Setup
9. Cost & Performance Optimization Recommendations